K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Basics of Neural Network & Deep Learning Lab ENCA354

LAB FILE

SUBMITTED BY:

Aashika Garg
2301201178
BCA (AI&DS)
SECTION -C

SUBMITTED TO:

Mr. Anoja Mishra

INDEX

SR. NO.	EXPERIMENTS	PAGE NO.	TEACHER SIGN
01.	Neural Network using TensorFlow (Low-Level API)	3-4	
02.	Neural Network using Basic Python (From Scratch)	5-6	
03.	Neural Network using Keras	7-8	
04.	Agriculture Price Prediction (Regression Problem)	9-10	
05.	MNIST Digit Classification using ANN	11-12	
06.	Plant Disease Detection using CNN	13-14	

Projects: Basics of Neural Network and Deep Learning

1.Neural Network using TensorFlow (Low-Level API)

- Build a simple feedforward neural network using **TensorFlow 1.x style (placeholders, sessions)**
- Understand computation graph and manual weight initialization

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
```

```
# Input placeholders
x = tf.placeholder(tf.float32, shape=[None, 2])
y = tf.placeholder(tf.float32, shape=[None, 1])

# Weights and biases
W1 = tf.Variable(tf.random_normal([2, 4]))
b1 = tf.Variable(tf.zeros([4]))

W2 = tf.Variable(tf.random_normal([4, 1]))
b2 = tf.Variable(tf.zeros([1]))
```

```
# Hidden layer
hidden = tf.nn.relu(tf.matmul(x, W1) + b1)

# Output layer
output = tf.matmul(hidden, W2) + b2

# Loss
loss = tf.reduce_mean(tf.square(output - y))

# Optimizer
optimizer = tf.train.GradientDescentOptimizer(learning_rate=0.01)
train = optimizer.minimize(loss)

# Initialize variables
init = tf.global_variables_initializer()
```

```python
# Dummy data
X_data = [[0,0], [0,1], [1,0], [1,1]]
Y_data = [[0], [1], [1], [0]]
```

```python
# Run session
with tf.Session() as sess:
    sess.run(init)

    for i in range(1000):
        _, l = sess.run([train, loss], feed_dict={x: X_data, y: Y_data})
        if i % 100 == 0:
            print("Loss:", l)

    print("Output:")
    print(sess.run(output, feed_dict={x: X_data}))
```

Final Output :

```
Loss: 1.042293
Loss: 0.2880014
Loss: 0.250013
Loss: 0.25000024
Loss: 0.25
Loss: 0.25
Loss: 0.25
Loss: 0.25
Loss: 0.25
Output:
[[0.50000143]
 [0.50000143]
 [0.50000143]
 [0.50000143]]
```

2.Neural Network using Basic Python (From Scratch)

```
import numpy as np

# Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Input (XOR problem)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

y = np.array([[0],
              [1],
              [1],
              [0]])
```

```
# Initialize weights
np.random.seed(1)
W1 = np.random.rand(2, 4)
b1 = np.zeros((1, 4))

W2 = np.random.rand(4, 1)
b2 = np.zeros((1, 1))
```

- Implement a neural network using **NumPy only**
- Include:
    - Forward propagation

```
# Training
learning_rate = 0.1
epochs = 10000

for i in range(epochs):

    # 🔹 Forward Propagation
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    # 🔹 Loss (Mean Squared Error)
    error = y - final_output
```

○ Backpropagation

```
# ◆ Backpropagation
d_output = error * sigmoid_derivative(final_output)

error_hidden = d_output.dot(W2.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_output)
```

○ Gradient descent

```
# ◆ Gradient Descent (update weights)
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

W1 += X.T.dot(d_hidden) * learning_rate
b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

# Print loss
if i % 1000 == 0:
    loss = np.mean(np.square(error))
    print(f"Epoch {i}, Loss: {loss}")
```

Final Output :

```
Epoch 0, Loss: 0.3180669563755088
Epoch 1000, Loss: 0.24993769944629102
Epoch 2000, Loss: 0.24975583638089172
Epoch 3000, Loss: 0.24905947343269236
Epoch 4000, Loss: 0.24403916846427992
Epoch 5000, Loss: 0.21081982683109768
Epoch 6000, Loss: 0.13953395652708653
Epoch 7000, Loss: 0.04296753063942452 4
Epoch 8000, Loss: 0.01598822354783635
Epoch 9000, Loss: 0.00880130931998014

Final Predictions:
[[0.0786917 ]
 [0.92546644]
 [0.92717202]
 [0.07974492]]
```

3. Neural Network using Keras

a. Build a model using **Keras Sequential API**
b. Train on a small dataset (e.g., Iris or dummy data)
c. Visualize accuracy and loss

```
import numpy as np
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)
```

```
# One-hot encoding
encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

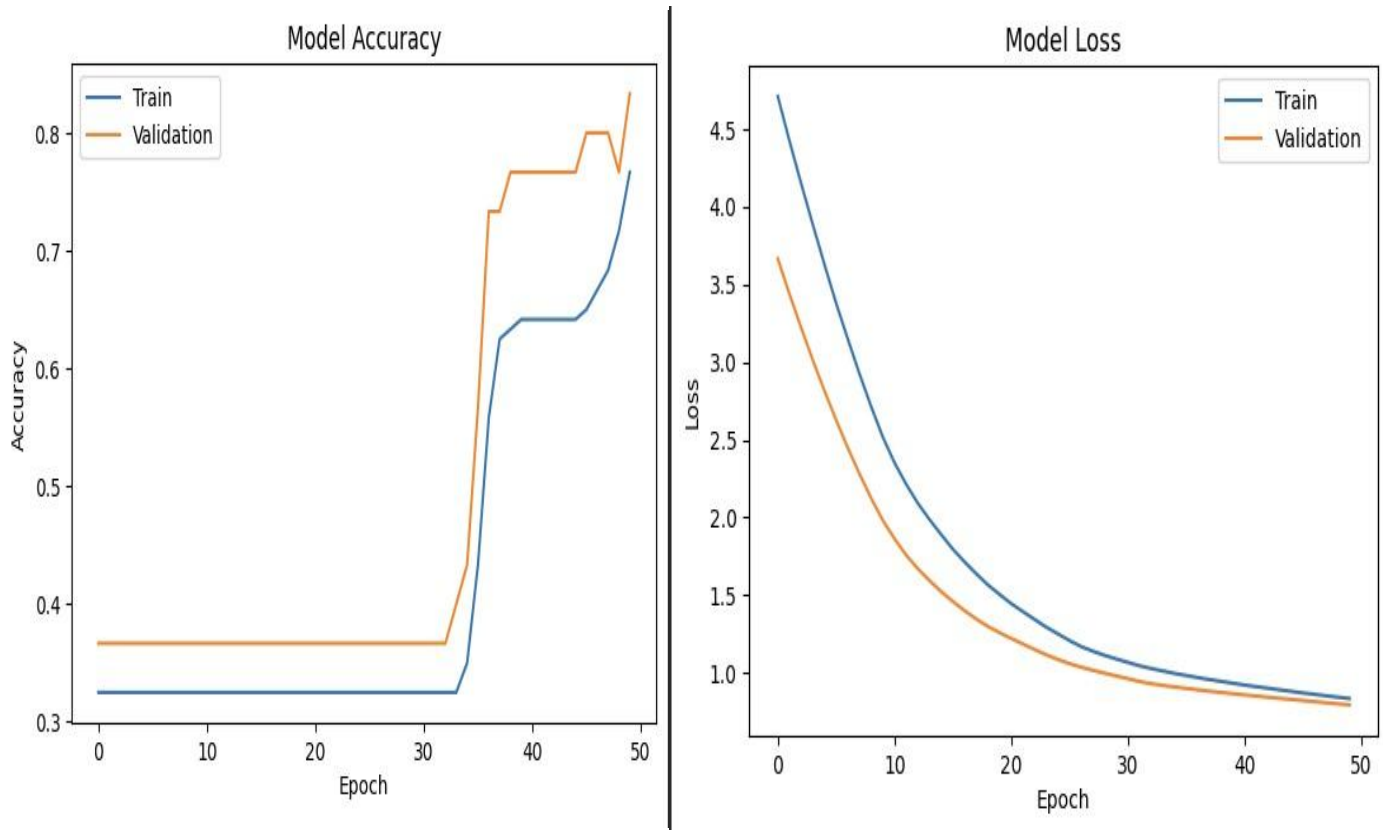
```
# Build model (Sequential API)
model = Sequential()
model.add(Dense(10, input_dim=4, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

```
# Compile model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
# Train model
history = model.fit(X_train, y_train, epochs=50, validation_data=(X_test, y_test))
```

```
import matplotlib.pyplot as plt

# Accuracy graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Loss graph
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

Final Output :



4. Agriculture Price Prediction (Regression Problem)

a. Take any dataset from GitHub (e.g., crop price dataset)
b. Build a regression model using ANN
c. Evaluate using:
   1.MSE
   2.R^2 Score

```
import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

```
import kagglehub
import pandas as pd
import os

# Download dataset
path = kagglehub.dataset_download("himanshu9648/crop-price-prediction-dataset

print("Dataset path:", path)

# Check files inside folder
files = os.listdir(path)
print("Files:", files)

# Load dataset (FIRST CSV FILE)
data = pd.read_csv(os.path.join(path, files[0]))

# Preview
print(data.head())
print(data.columns)
```

```
Using Colab cache for faster access to the 'crop-price-prediction-dataset-in-
Dataset path: /kaggle/input/crop-price-prediction-dataset-in-india
Files: ['crop_price_dataset.csv']
         month commodity_name  avg_modal_price  avg_min_price  avg_max_price
0  2025-03-01           Maize          2341.58        2191.23        2402.98
1  2025-02-01           Maize          2356.01        2202.35        2414.46
2  2025-01-01           Maize          2382.20        2221.31        2448.22
3  2024-12-01           Maize          2328.78        2164.86        2411.66
4  2024-11-01           Maize          2273.46        2077.63        2377.78

  state_name district_name calculationType  change
0      India            All          Monthly  -14.43
1      India            All          Monthly  -26.19
2      India            All          Monthly   53.42
3      India            All          Monthly   55.32
4      India            All          Monthly   35.48
Index(['month', 'commodity_name', 'avg_modal_price', 'avg_min_price',
       'avg_max_price', 'state_name', 'district_name', 'calculationType',
       'change'],
      dtype='object')
```

```python
# Detect target column
price_cols = [col for col in data.columns if "price" in col.lower()]
target = price_cols[0]   # first matching column

print("Using target:", target)

X = data.drop(target, axis=1)
y = data[target]

# Encoding
X = pd.get_dummies(X)

# Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
Using target: avg_modal_price
```

```python
# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```python
# Build ANN model
model = Sequential()
model.add(Dense(16, activation='relu', input_dim=X.shape[1]))
model.add(Dense(8, activation='relu'))
model.add(Dense(1))  # Regression output
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```python
model.compile(optimizer='adam', loss='mse')

# Train
model.fit(X_train, y_train, epochs=50, verbose=1)

# Predict
y_pred = model.predict(X_test)

# 🔥 FIX for safety
y_test = y_test.values   # ensure numpy format

# Evaluation
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R2 Score:", r2)
```

Final Output :

```
MSE: 3523714.898985573
R2 Score: 0.8692720080064275
```

5. MNIST Digit Classification using ANN

a. Load MNIST dataset
b. Build a simple ANN (fully connected network)
c. Evaluate accuracy and confusion matrix

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tens
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
# Normalize data (0-1 range)
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```
#  Build ANN model (Fully Connected)
model = Sequential()

model.add(Flatten(input_shape=(28, 28)))  # convert 2D → 1D
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))  # 10 digits (0-9)

# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

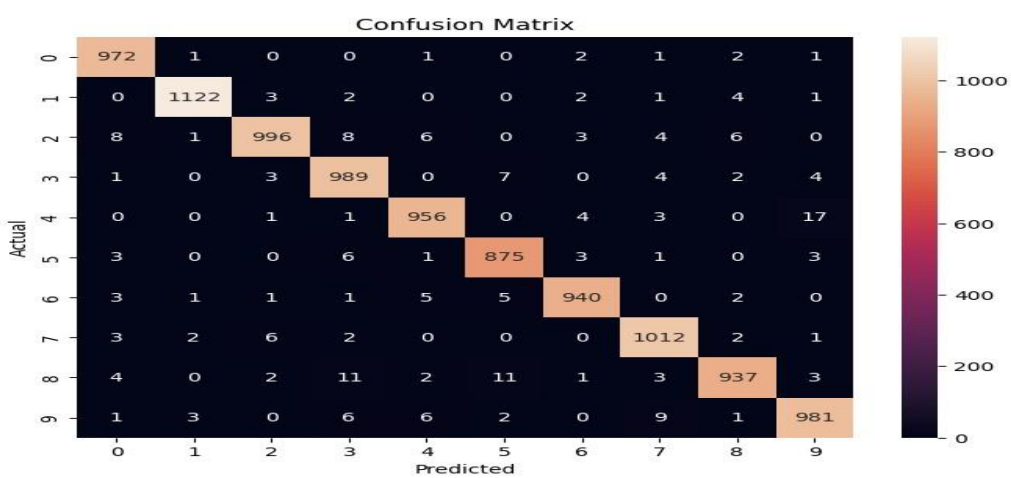
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/resha
  super().__init__(**kwargs)
```

```python
# Train
history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))

#  Evaluate Accuracy
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

#  Predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.9287 - loss: 0.2415 - val_accuracy: 0.9603 - val_loss: 0.1301
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - accuracy: 0.9682 - loss: 0.1025 - val_accuracy: 0.9696 - val_loss: 0.1001
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9767 - loss: 0.0746 - val_accuracy: 0.9758 - val_loss: 0.0796
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9821 - loss: 0.0551 - val_accuracy: 0.9758 - val_loss: 0.0829
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9861 - loss: 0.0438 - val_accuracy: 0.9780 - val_loss: 0.0684
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9780 - loss: 0.0684
Test Accuracy: 0.9779999852180481
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

```python
# ✅ Confusion Matrix
cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Final Output :



6. Plant Disease Detection using CNN

a. Use plant leaf dataset (e.g., from Kaggle)
b. Multi-class classification of diseases
c. Save basic prediction model

```
import kagglehub
import os

# Download dataset
path = kagglehub.dataset_download("naolbm/plant-leaf-dataset")

print("Dataset path:", path)

# Check folders (classes)
print(os.listdir(path))
```

```
Using Colab cache for faster access to the 'plant-leaf-dataset' dataset.
Dataset path: /kaggle/input/plant-leaf-dataset
['Coffee', 'Maize']
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

img_size = (128, 128)
batch_size = 32

datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    path,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    path,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation'
)
```

```
Found 4683 images belonging to 2 classes.
Found 1169 images belonging to 2 classes.
```

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential()

model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(2,2))

model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dense(train_data.num_classes, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```python
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=5
)
```

```
Epoch 1/5
147/147 ━━━━━━━━━━━━━━━━━━━━ 147s 987ms/step - accuracy: 0.9667 - loss: 0.1332 - val_accuracy: 0.9966 - val_loss: 0.0157
Epoch 2/5
147/147 ━━━━━━━━━━━━━━━━━━━━ 139s 945ms/step - accuracy: 0.9974 - loss: 0.0092 - val_accuracy: 0.9966 - val_loss: 0.0237
Epoch 3/5
147/147 ━━━━━━━━━━━━━━━━━━━━ 148s 1s/step - accuracy: 0.9957 - loss: 0.0206 - val_accuracy: 1.0000 - val_loss: 3.8901e-04
Epoch 4/5
147/147 ━━━━━━━━━━━━━━━━━━━━ 149s 1s/step - accuracy: 0.9996 - loss: 0.0010 - val_accuracy: 1.0000 - val_loss: 1.6583e-04
Epoch 5/5
147/147 ━━━━━━━━━━━━━━━━━━━━ 145s 988ms/step - accuracy: 1.0000 - loss: 9.7136e-05 - val_accuracy: 1.0000 - val_loss: 1.9015e-05
```

```python
model.save("plant_disease_model.h5")
```

```python
# Load image
img = image.load_img(img_path, target_size=(128,128))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Prediction
prediction = model.predict(img_array)
predicted_index = np.argmax(prediction)

# Output
print("Predicted class index:", predicted_index)
print("Predicted class name:", class_names[predicted_index])
```

Final Output :

```
Classes: ['Coffee', 'Maize']
Using image: /kaggle/input/plant-leaf-dataset/Coffee/data/Miner/1269.jpg
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 751ms/step
Predicted class index: 0
Predicted class name: Coffee
```